

Objetivos da aula:

- Compreender o conceito e a finalidade das views em bancos de dados relacionais;
- Criar, alterar e remover views no PostgreSQL;
- Identificar views atualizáveis e não atualizáveis;
- Compreender o funcionamento e o uso de materialized views;
- Avaliar quando utilizar views tradicionais ou materialized views, considerando desempenho e manutenção.

Índice:

- i. Introdução às Views
- ii. Criação, Manutenção e Atualização de Views
- iii. Materialized Views

i. Introdução às Views**1.1. Introdução**

Em bancos de dados relacionais, é comum que consultas SQL se tornem complexas, longas e repetitivas, especialmente quando envolvem JOIN, filtros elaborados e regras de negócio específicas. Para lidar com esse problema, os SGBDs oferecem o conceito de views.

Uma view pode ser entendida como uma tabela virtual, definida a partir de uma consulta SQL, cujo resultado pode ser reutilizado como se fosse uma tabela comum.

1.2. O que é um View

Uma view é um objeto do banco de dados que armazena uma consulta SQL, e não os dados propriamente ditos.

Características fundamentais:

- Não armazena dados fisicamente (na maioria dos casos);
- Sempre reflete o estado atual das tabelas base;
- Pode ser consultada com SELECT como qualquer tabela;
- Serve como uma camada de abstração sobre os dados.

Definição conceitual:

Uma view é uma consulta armazenada no banco de dados, apresentada ao usuário como uma tabela virtual.

1.3. Motivação para o uso de Views**1.3.1 Simplificação de consultas**

Consultas complexas podem ser encapsuladas em uma view, facilitando o uso posterior.

```
SELECT p.nome, c.placa, c.ano  
FROM pessoa p
```

```
JOIN carro c ON p.id_pessoa = c.id_pessoa  
WHERE c.ano >= 2020;
```

Essa consulta pode ser transformada em uma view, evitando repetição e erros.

1.3.2 Reutilização de código SQL

Uma vez criada, a view pode ser reutilizada em diferentes consultas:

```
SELECT *  
FROM vw_carros_recentes;
```

Isso melhora a manutenibilidade do banco de dados.

1.3.3 Segurança e controle de acesso

Views podem ser usadas para:

- ocultar colunas sensíveis;
- restringir linhas visíveis a determinados usuários;
- conceder acesso indireto às tabelas base.

Exemplo:

- Usuários podem consultar uma view sem ter permissão direta sobre a tabela original.

1.3.4 Padronização de Acesso aos Dados

Views ajudam a garantir que todos os usuários e aplicações:

- consultem os dados da mesma forma;
- utilizem as mesmas regras de negócio;
- evitem consultas inconsistentes.

1.4. View × Tabela

É importante distinguir claramente uma view de uma tabela física.

Aspecto	Tabela	View
Armazena dados	Sim	Não (em geral)
Ocupa espaço	Sim	Não
Pode ser atualizada diretamente	Sim	Depende
Baseada em consulta	Não	Sim

A view depende sempre das tabelas base para existir e funcionar corretamente.

1.5. Views e atualização de dados

Nem toda view permite operações de modificação (INSERT, UPDATE, DELETE).

De forma geral:

- Views simples (baseadas em uma única tabela, sem agregações) **podem ser atualizáveis**;
- Views complexas (com JOIN, GROUP BY, DISTINCT, funções agregadas) **não são atualizáveis automaticamente**.

Esse aspecto será aprofundado nos próximos capítulos.

1.6. Considerações Importantes

- Uma view **não melhora desempenho automaticamente**;
- Cada consulta à view executa a consulta subjacente;
- Views facilitam leitura e organização, não substituem índices;
- Alterações na estrutura das tabelas base podem invalidar views.

Esses pontos reforçam que views são uma ferramenta de **modelagem lógica**, e não de otimização por si só.

ii. Criação, Manutenção e Atualização de Views

2.1. Comando CREATE VIEW

A criação de uma view é feita com o comando CREATE VIEW, que associa um nome a uma consulta SQL.

Exemplo:

```
CREATE VIEW vw_carros_recentes AS
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE c.ano >= 2020;
```

Após criada, a view pode ser utilizada como uma tabela:

```
SELECT *
FROM vw_carros_recentes;
```

2.2. Comando CREATE OR REPLACE VIEW

O comando CREATE OR REPLACE VIEW permite **alterar a definição de uma view existente** sem precisar removê-la previamente.

```
CREATE OR REPLACE VIEW vw_carros_recentes AS
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE c.ano >= 2022;
```

Vantagens:

- evita erros de dependência;

- preserva permissões concedidas à view;
- facilita manutenção e evolução do banco.

2.3. Comando DROP VIEW

Para remover uma view do banco de dados, utiliza-se o comando DROP VIEW.

```
DROP VIEW vw_carros_recentes;
```

Para evitar erro caso a view não exista:

```
DROP VIEW IF EXISTS vw_carros_recentes;
```

Importante:

- A remoção da view não afeta as tabelas base;
- Views dependentes podem impedir a remoção, exigindo CASCADE.

```
DROP VIEW vw_carros_recentes CASCADE;
```

2.4. Comando para listar as views

Lista todas as views do schema public.

```
SELECT viewname, definition  
FROM pg_views  
WHERE schemaname = 'public';
```

2.5. Views Atualizáveis

Uma view é considerada atualizável quando permite operações de:

- INSERT
- UPDATE
- DELETE

2.5.1 Regras gerais

No PostgreSQL, uma view tende a ser atualizável quando:

- é baseada em uma única tabela;
- não utiliza JOIN;
- não possui GROUP BY, DISTINCT, agregações ou subconsultas complexas.

2.5.2 Exemplo de view atualizável

```
CREATE VIEW vw_pessoa_simples AS  
SELECT id_pessoa, nome, nascimento  
FROM pessoa;
```

Operação válida:

```
UPDATE vw_pessoa_simples
SET nome = 'Pedro Prado Atualizado'
WHERE id_pessoa = 10;
```

Essa operação será refletida diretamente na tabela `pessoa`.

2.5.3 Exemplo de view não atualizável

```
CREATE VIEW vw_pessoa_carro AS
SELECT p.nome, c.placa
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa;
```

Tentativas de INSERT, UPDATE ou DELETE nessa view **gerarão erro**, pois a view envolve múltiplas tabelas.

2.6. Limitações das views

Algumas limitações importantes devem ser consideradas:

- Views **não armazenam dados** (salvo materialized views);
- Views complexas **não são atualizáveis automaticamente**;
- Alterações na estrutura das tabelas base podem invalidar views;
- Views **não substituem índices** nem melhoram desempenho por si só.

Essas limitações reforçam que views são um recurso de **organização e abstração**, e não de otimização direta.

iii. Materialized Views

3.1. Introdução

Nos capítulos anteriores, foram estudadas as views tradicionais, que funcionam como consultas armazenadas e são executadas sempre que acessadas.

Embora esse modelo seja adequado para organização e abstração, ele não resolve problemas de desempenho em consultas complexas ou sobre grandes volumes de dados.

Para esses cenários, o PostgreSQL disponibiliza as materialized views, que combinam características de views e tabelas físicas.

3.2. O que é uma Materialized View

Uma materialized view é um objeto do banco de dados que:

- armazena fisicamente o resultado de uma consulta;
- funciona como uma tabela derivada;
- precisa ser explicitamente atualizada para refletir alterações nas tabelas base.

Definição conceitual:

Uma materialized view é uma consulta armazenada cujo resultado é persistido em disco.

Diferentemente das views tradicionais, a consulta não é reexecutada a cada acesso.

3.3. Motivação para o uso de Materialized Views

As materialized views são indicadas quando:

- a consulta é complexa (múltiplos JOIN, agregações, subconsultas);
- o volume de dados é elevado;
- os dados não precisam estar atualizados em tempo real;
- há necessidade de ganho de desempenho em consultas repetidas.

Exemplos típicos de uso:

- relatórios gerenciais;
- painéis analíticos (*dashboards*);
- consolidações periódicas de dados;
- consultas analíticas de leitura intensiva.

3.4. View × Materialized View

Aspecto	View	Materialized View
Armazena dados fisicamente	Não	Sim
Executa consulta a cada acesso	Sim	Não
Precisa de atualização manual	Não	Sim
Pode melhorar desempenho	Não	Sim
Ocupa espaço em disco	Não	Sim

Essa comparação deixa claro que materialized views são um recurso de desempenho, enquanto views tradicionais são um recurso de abstração.

3.5. Criação de Materialized View

A criação é feita com o comando CREATE MATERIALIZED VIEW.

```
CREATE MATERIALIZED VIEW mv_carros_recentes AS
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE c.ano >= 2020;
```

Comparação de resultados usando view tradicional x materialized view:

```
EXPLAIN ANALYSE
SELECT *
FROM mv_carros_recentes;
```

```
EXPLAIN ANALYSE
SELECT *
FROM vw_carros_recentes;
```

"Seq Scan on mv_carros_recentes ..."
"Execution Time: 0.992 ms"

"Hash Join ..."
"Hash Cond: (c.id_pessoa = p.id_pessoa)"
" -> Seq Scan on carro c"
" -> Hash "
" -> Seq Scan on pessoa p "
"Execution Time: 53.960 ms"

3.6 Atualização de Materialized Views (REFRESH)

Diferentemente das views tradicionais, a materialized view **não é atualizada automaticamente**.

Para atualizar seu conteúdo, utiliza-se:

```
REFRESH MATERIALIZED VIEW mv_carros_recentes;
```

Após o REFRESH, os dados passam a refletir o estado atual das tabelas base.

3.6.1 REFRESH MATERIALIZED VIEW CONCURRENTLY

O PostgreSQL oferece a opção de atualização concorrente:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_carros_recentes;
```

Características:

- permite consultas simultâneas durante o refresh;
- exige a existência de um **índice único** na materialized view;
- costuma ser mais lenta que o refresh tradicional.

3.7 Índices em Materialized Views

Uma materialized view pode (e deve) possuir índices próprios.

```
CREATE INDEX idx_mv_carros_nome  
ON mv_carros_recentes (nome);
```

Vantagens:

- consultas sobre a materialized view tornam-se ainda mais rápidas;
- comportamento semelhante ao de uma tabela física.

Esse recurso torna materialized views extremamente eficazes em cenários analíticos.

3.8 Atualização e modificação de dados

Materialized views:

- não são atualizáveis diretamente com INSERT, UPDATE ou DELETE;
- qualquer modificação deve ser feita nas tabelas base;
- o resultado só será refletido após um REFRESH.

Esse comportamento garante consistência, mas exige planejamento sobre frequência de atualização.

3.9. Limitações e cuidados

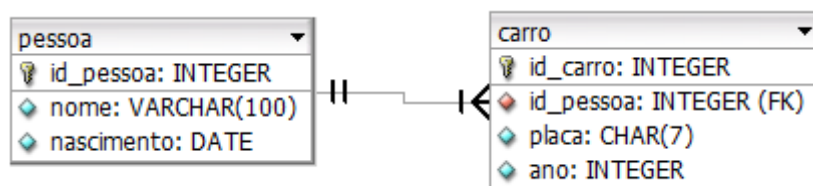
Alguns pontos importantes:

- ocupam espaço em disco;
- podem ficar desatualizadas;
- exigem controle de quando executar o REFRESH;
- não substituem índices nas tabelas base;
- aumentam a complexidade da manutenção do banco.

Assim, o uso de materialized views deve ser **criterioso e justificado**.

Exercícios

Para a correta execução dos exercícios, utilize os comandos a seguir para criar as tabelas e carregar os registros a partir de arquivos CSV.



Cada arquivo contém 100 mil registros.

Antes de executar os comandos COPY, é necessário **ajustar o caminho dos arquivos** de acordo com a localização dos CSVs no seu computador.

```
DROP TABLE IF EXISTS carro, pessoa
CASCADE;

CREATE TABLE IF NOT EXISTS pessoa (
    id_pessoa INTEGER PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    nascimento DATE
);

CREATE TABLE IF NOT EXISTS carro (
    id_carro INTEGER PRIMARY KEY,
    placa CHAR(7) NOT NULL UNIQUE,
    ano INTEGER,
    id_pessoa INTEGER NOT NULL,
```

```
COPY    pessoa    (id_pessoa,    nome,
nascimento)
FROM 'C:/caminho/aula3_pessoa.csv'
DELIMITER ','
CSV HEADER;

COPY    carro    (id_carro,    placa,    ano,
id_pessoa)
FROM 'C:/caminho/aula3_carro.csv'
DELIMITER ','
CSV HEADER;
```



```
FOREIGN KEY (id_pessoa)
REFERENCES pessoa(id_pessoa)
ON DELETE CASCADE
);
```

Exercício 1 – Criação de view simples

Enunciado:

Crie uma view simples que liste o id_pessoa, o nome e a data de nascimento de todas as pessoas cadastradas.

Tarefas:

1. Crie a view com um nome apropriado.
2. Execute um SELECT sobre a view para validar o resultado.
3. Explique por que essa view pode ser considerada atualizável.

Resposta:

1. Crie a view com um nome apropriado.

```
CREATE VIEW vw_pessoa_simples AS
SELECT id_pessoa, nome, nascimento
FROM pessoa;
```

2. Execute um SELECT sobre a view para validar o resultado.

```
SELECT *
FROM vw_pessoa_simples;
```

3. Explique por que essa view pode ser considerada atualizável.

Essa view é considerada **atualizável** porque:

- é baseada em **uma única tabela** (pessoa);
- não utiliza JOIN;
- não possui agregações (GROUP BY);
- não utiliza DISTINCT, subconsultas ou funções de janela;
- cada linha da view corresponde diretamente a **uma única linha da tabela base**.

Dessa forma, o PostgreSQL consegue mapear de forma inequívoca operações como UPDATE e DELETE na view para a tabela pessoa.

Exemplo de operação válida:

```
UPDATE vw_pessoa_simples
SET nome = 'Pedro Prado Exer01'
WHERE id_pessoa = 10;
```

Essa atualização será refletida diretamente na tabela pessoa.

Exercício 2 – View com JOIN

Enunciado:

Crie uma view que liste o nome da pessoa, a placa do carro e o ano do carro, considerando apenas veículos fabricados a partir de 2018.

Tarefas:

1. Crie a view utilizando JOIN entre as tabelas pessoa e carro.
2. Execute um SELECT sobre a view.
3. Explique por que essa view **não é atualizável**.

Resposta:

1. Crie a view utilizando JOIN entre as tabelas pessoa e carro.

```
CREATE VIEW vw_pessoa_carro_recente AS
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE c.ano >= 2018;
```

2. Execute um SELECT sobre a view.

```
SELECT *
FROM vw_pessoa_carro_recente;
```

3. Explique por que essa view **não é atualizável**.

Essa view **não é atualizável automaticamente** no PostgreSQL porque:

- é baseada em **mais de uma tabela** (pessoa e carro);
- utiliza um **JOIN**, o que impede o mapeamento direto de uma linha da view para uma única linha em uma tabela base;
- o PostgreSQL não consegue determinar, de forma inequívoca, como propagar operações de INSERT, UPDATE ou DELETE para as tabelas envolvidas.

Por esse motivo, tentativas de modificação direta resultarão em erro, como no exemplo:

```
UPDATE vw_pessoa_carro_recente
SET ano = 2020
WHERE placa = 'ABC1234';
```

Essa operação não é permitida, pois a view envolve múltiplas tabelas.

Exercício 3 – Alteração de view (CREATE OR REPLACE)

Enunciado:

A view criada no Exercício 2 deve ser alterada para considerar apenas carros fabricados a partir de **2020**.

Tarefas:

1. Utilize CREATE OR REPLACE VIEW para modificar a definição da view.

2. Verifique se a alteração foi aplicada corretamente.
3. Explique a vantagem de utilizar CREATE OR REPLACE VIEW em vez de DROP VIEW seguido de CREATE VIEW.

Resposta:

1. Utilize CREATE OR REPLACE VIEW para modificar a definição da view.

```
CREATE OR REPLACE VIEW vw_pessoa_carro_recente AS
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE c.ano >= 2020;
```

2. Verifique se a alteração foi aplicada corretamente.

```
SELECT *
FROM vw_pessoa_carro_recente;
```

3. Explique a vantagem de utilizar CREATE OR REPLACE VIEW em vez de DROP VIEW seguido de CREATE VIEW.
 - evita a remoção temporária da view;
 - preserva as **permissões** concedidas à view;
 - reduz o risco de erros em objetos dependentes;
 - facilita a **manutenção e evolução** do banco de dados.

Exercício 4 – Remoção de view

Enunciado:

Remova a view criada no Exercício 2.

Tarefas:

1. Exclua a view utilizando um comando que evite erro caso a view não exista.
2. Por que a remoção da view não afeta as tabelas base?

Resposta:

1. Exclua a view utilizando um comando que evite erro caso a view não exista.

```
DROP VIEW IF EXISTS vw_pessoa_carro_recente;
```

2. Por que a remoção da view não afeta as tabelas base?

A remoção da view não afeta as tabelas pessoa e carro porque:

- uma view armazena apenas a definição da consulta (isto é, o SQL);
- ela não armazena os dados fisicamente (diferentemente de uma materialized view);
- portanto, ao executar DROP VIEW, apenas o objeto “view” é removido, enquanto as tabelas e seus dados permanecem intactos.

Exercício 5 – Listagem de views

Enunciado:

Elabore uma consulta SQL que liste todas as views existentes no schema public.

Tarefas:

1. Listar as views do schema public usando o catálogo pg_views.
2. Listar as views usando o padrão information_schema.
3. Explique a diferença entre usar pg_views e information_schema.views.

Resposta:

1. Listar as views do schema public usando o catálogo pg_views.

```
SELECT viewname, definition
FROM pg_views
WHERE schemaname = 'public'
ORDER BY viewname;
```

2. Listar as views usando o padrão information_schema.

```
SELECT table_name, view_definition
FROM information_schema.views
WHERE table_schema = 'public'
ORDER BY table_name;
```

3. Explique a diferença entre usar pg_views e information_schema.views.

- pg_views
 - É um catálogo específico do PostgreSQL (mais “nativo” do SGBD).
 - Em geral, é mais direto para administração e inspeção do banco.
 - Mostra a definição na coluna definition.
- information_schema.views
 - Faz parte do padrão SQL (mais portátil entre SGBDs).
 - Pode variar em nível de detalhe dependendo do SGBD.
 - Fornece informações padronizadas sobre views (como table_schema, table_name, view_definition).

Exercício 6 – Criação de Materialized View

Enunciado:

Crie uma materialized view que liste o nome da pessoa e a quantidade de carros que ela possui.

Tarefas:

1. Crie a materialized view utilizando agregação.
2. Execute um SELECT para verificar o conteúdo.
3. Explique por que esse tipo de consulta é adequado para uma materialized view.

Resposta:

1. Crie a materialized view utilizando agregação.

```
CREATE MATERIALIZED VIEW mv_qtd_carros_pessoa AS
SELECT
    p.id_pessoa,
    p.nome,
    COUNT(c.id_carro) AS quantidade_carros
FROM pessoa p
LEFT JOIN carro c ON p.id_pessoa = c.id_pessoa
GROUP BY p.id_pessoa, p.nome;
```

2. Execute um SELECT para verificar o conteúdo.

```
SELECT *
FROM mv_qtd_carros_pessoa;
```

3. Explique por que esse tipo de consulta é adequado para uma materialized view.

- envolve JOIN entre tabelas;
- utiliza agregação (COUNT), que pode ser custosa em grandes volumes de dados;
- tende a ser usada em relatórios e consultas analíticas;
- não exige atualização em tempo real a cada acesso.

Ao materializar o resultado:

- o custo do JOIN e da agregação é pago apenas no momento do REFRESH;
- consultas subsequentes tornam-se significativamente mais rápidas.

Exercício 7 – Atualização de Materialized View

Enunciado:

Após a criação da materialized view do Exercício 6, considere que novos registros foram inseridos na tabela carro.

Tarefas:

1. Execute o comando necessário para atualizar o conteúdo da materialized view.
2. Verifique se os dados foram atualizados.
3. Explique a diferença entre uma view tradicional e uma materialized view quanto à atualização dos dados.

Resposta:

1. Execute o comando necessário para atualizar o conteúdo da materialized view.

```
REFRESH MATERIALIZED VIEW mv_qtd_carros_pessoa;
```

2. Verifique se os dados foram atualizados.

```
SELECT *
FROM mv_qtd_carros_pessoa;
```

3. Explique a diferença entre uma view tradicional e uma materialized view quanto à atualização dos dados.

- View tradicional

- Não armazena dados;
- Sempre reflete o estado atual das tabelas base;
- A consulta é reexecutada a cada acesso;
- Não requer comando de atualização.
- Materialized view
 - Armazena os dados fisicamente;
 - Não reflete automaticamente alterações nas tabelas base;
 - Requer a execução explícita de REFRESH MATERIALIZED VIEW;
 - Oferece melhor desempenho em consultas repetitivas e complexas.

Exercício 8 – REFRESH CONCURRENTLY

Enunciado:

Recrie a materialized view do Exercício 6 de forma que permita atualização concorrente.

Tarefas:

1. Crie um índice único necessário para permitir o uso do REFRESH CONCURRENTLY.
2. Execute o comando REFRESH MATERIALIZED VIEW CONCURRENTLY.
3. Explique as vantagens e limitações desse tipo de atualização.

Resposta:

1. Crie um índice único necessário para permitir o uso do REFRESH CONCURRENTLY.

Para que uma materialized view possa ser atualizada de forma concorrente, o PostgreSQL exige a existência de um índice único que identifique unicamente cada linha.

No caso da materialized view mv_qtd_carros_pessoa, o índice único pode ser criado sobre a coluna id_pessoa, pois ela identifica unicamente cada pessoa no resultado:

```
CREATE UNIQUE INDEX idx_mv_qtd_carros_pessoa_id
ON mv_qtd_carros_pessoa (id_pessoa);
```

Sem esse índice, o comando REFRESH MATERIALIZED VIEW CONCURRENTLY não pode ser executado.

2. Execute o comando REFRESH MATERIALIZED VIEW CONCURRENTLY.

Após a criação do índice único, a atualização concorrente pode ser realizada com:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_qtd_carros_pessoa;
```

Esse comando atualiza o conteúdo da materialized view sem bloquear consultas de leitura, permitindo que outros usuários continuem acessando os dados durante o processo.

3. Explique as vantagens e limitações desse tipo de atualização.

O uso de REFRESH MATERIALIZED VIEW CONCURRENTLY apresenta as seguintes vantagens:

- permite que a materialized view continue sendo consultada durante a atualização;
- evita bloqueios exclusivos de leitura;

- é adequado para ambientes com múltiplos usuários e consultas frequentes.

Esse comportamento é especialmente importante em sistemas analíticos e ambientes de produção.

Apesar das vantagens, esse tipo de atualização possui algumas limitações:

- é mais lenta que o REFRESH tradicional;
- exige obrigatoriamente um índice único;
- não pode ser executada dentro de uma transação (BEGIN ... COMMIT);
- consome mais recursos durante o processo.

Por esse motivo, seu uso deve ser avaliado conforme o contexto da aplicação.